

MARC — Architectures, Visually Explained

Read this if you've never seen the project. It explains, in pictures and plain English, the three networks we compare and why the third one is the point of the whole thesis. Companion files: *01-vanilla-ippo*, *02-marc-architecture*, *03-marc-fixed-aux*.

The 30-second version

We train cooperative agents (Overcooked kitchens; MPE *spread* — N agents must each cover a different landmark and *not* bunch up). Plain multi-agent PPO (“vanilla”) has a classic failure: **redundant / lazy agents** — two do the same job, one landmark sits empty. MARC’s bet: give each agent a small “who am I / who are my teammates” module, and optionally a loss that pushes teammates into *different* roles.

#	network	one sentence
1	Vanilla IPPO	plain actor–critic; the baseline
2	MARC architecture	+ self-encoder, per-teammate inferencer, attention pool (no loss)
3	MARC + fixed aux	+ an align/diversity loss, normalized, gated and annealed

Headline: #2 beats #1 robustly everywhere. #3 beats #2 by a *small* amount that **grows with team size** (the on-thesis prediction) — once the loss is engineered correctly.

The colour key (used in every diagram)

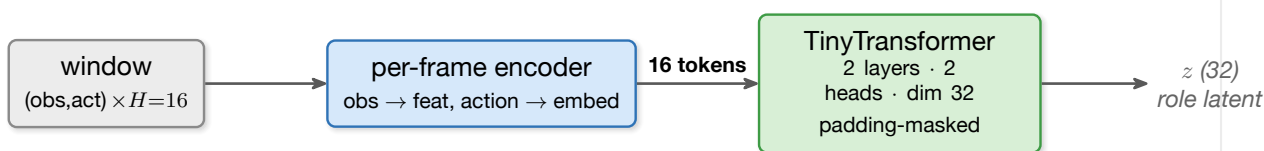
Each box is coloured by *what kind of component it is*. Same kind, same colour, everywhere.

INPUT raw env tensors **ENCODER** obs $\rightarrow$ feature vector **TRANSFORMER** history self-attention
X-ATTN cross-attention over M teammates **HEAD** policy / value MLP **LOSS** auxiliary term or gate

The shared vocabulary: one block for the whole project

Every MARC network is built from **one reusable block**, the **HistoryEncoder**: it turns a window of the last $H=16$ steps (what an agent saw and did) into one 32-number **role latent** z — a compressed answer to “*what role has this agent been playing lately?*” The whole project is just *how many copies you run and what you do with the z ’s*.

HistoryEncoder — the block reused everywhere



Combined scoreboard

MPE `simple_spread`, eval return (higher is better; reward is negative distance-to-landmarks, so -10 is far better coverage than -50). *arch* — *vanilla* and *aux gain* (= normgateup — arch) are the two effects we care about.

N	vanilla	MARC arch	MARC+aux	arch – vanilla	aux gain
3	–19.52	–10.95	–10.37	+8.57	+0.58
6	–34.69	–31.67	–29.31	+3.03	+2.36
9	–54.03	–43.42	–40.12	+10.60	+3.30

Two stories, cleanly separated: the **architecture** is the big, robust effect (beats vanilla at every team size, and on Overcooked’s coordination layouts too); the **aux loss** is a small *extra* gain on top that grows with N ($+0.58 \rightarrow +2.36 \rightarrow +3.80$) and tightens seed variance. Not variance: $N=3$ and $N=6$ are powered to 8 seeds and individually significant (paired $p=0.0013$ and 0.0002 , 8/8 positive); every paired run so far improved (19/19); the grows-with- N trend is monotone. Full statistics and per-seed tables in file 3.